Software Architecture Design for Among Us

Name: Syed Hassan Tahir

SAP ID: 59738

1. Game Overview

Among Us is a multiplayer social deduction game in which players are assigned the role of either Crewmates or Impostors. Crewmates complete tasks around a spaceship or map while trying to identify the Impostors. Impostors attempt to eliminate Crewmates and sabotage systems without being detected. The game involves real-time player interaction, task completion, meetings, voting, and network synchronization.

2. High-Level Architecture

Player System: Manages player profiles, roles, movement, status, and task progress.

Matchmaking/Lobby System: Handles room creation, joining, leaving, player readiness, and game start conditions.

Game State Management System: Controls transitions between Lobby, Gameplay, Meeting, Voting, and End Game states.

Task System: Assigns and tracks tasks for Crewmates and checks win conditions.

Voting System: Handles emergency meetings, vote collection, vote counting, and player ejection.

Kill & Sabotage System: Allows Impostors to perform eliminations and trigger sabotage events.

Networking System: Synchronizes game data between all connected clients and the server.

UI System: Displays menus, task lists, voting screens, notifications, and results.

3. Code Structure

Core Classes:
- GameManager
- PlayerManager
- Player
- LobbyManager

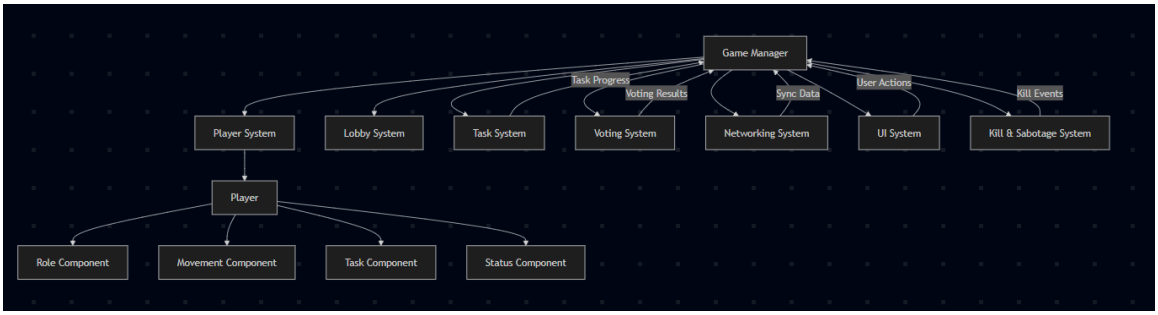
- TaskManager
- VotingManager
- SabotageManager
- NetworkManager
- UIManager

Components:
- MovementComponent
- TaskComponent
- RoleComponent
- Health/Status Component
- NetworkSync Component

Responsibilities:
GameManager controls game flow. PlayerManager manages player data. TaskManager handles tasks. VotingManager processes meetings and votes. NetworkManager synchronizes data. UIManager updates the interface.

Communication:
Systems communicate through events and manager classes. For example, TaskManager informs GameManager when all tasks are complete, and VotingManager informs GameManager after vote results are finalized.

4. Architecture Diagram



5. Design Decisions
A modular architecture was chosen because it separates responsibilities into independent systems. This improves maintainability, scalability, testing, and debugging. Manager-based architecture is commonly used in Unity projects and allows each subsystem to focus on a single responsibility. Event-driven communication reduces coupling between systems and improves code organization.

6. Conclusion

This architecture provides a clear and scalable foundation for implementing an Among Us style multiplayer game. By separating gameplay features into dedicated systems and managers, development becomes easier, more maintainable, and better suited for future expansion.